



全链路审计流程

让 AI Coding 行为可追溯、可复盘、可问责

HackProve World 2026 白帽世界大会中国·澳门

主题 09 03:03:09 - 03:25:29

本讲义依据本主题 transcript、rel/abs SRT 与官方参考池整理；外部资料仅限 official_references.md 所列条目。

目录

1	问题入口：AI Coding 的行为已经超过一次回答	2
2	首次定义：审计链路、Trace 与证据 E	2
2.1	审计链路	2
2.2	Trace	3
2.3	证据 E 的生命周期	3
3	全链路视图：从需求到发布的 Trace	3
4	审计对象：prompt、上下文、工具、diff、测试、发布、审批	4
4.1	Prompt：意图与边界的起点	4
4.2	上下文：模型看到的世界	4
4.3	工具：从“说”到“做”的跨越	7
4.4	Diff：工程事实的主证据	7
4.5	测试：把“可运行”变成证据	7
4.6	发布与审批：责任落点	8
5	黑匣子式追溯：把不可见的限制层摊平成平面	8
6	落地模板：字段、grep 词与报告格式	9
6.1	审计字段模板	9
6.2	grep 词与风险词	9
6.3	报告格式	9
7	官方参考如何支撑本主题	10
8	结语：从单点审核到工作流审计	10

1 问题入口：AI Coding 的行为已经超过一次回答

本主题从一个很朴素的现场判断开始：日常使用 AI Coding 产品时，安全研究者经常遇到两类阻断。第一类来自模型层，模型把安全研究请求误判为危险请求，导致演示、样本分析、漏洞复现说明被提前截断。第二类来自客户端层，AI Coding 客户端自身的授权、沙箱、策略、工具放行机制会切断整个安全研究工作流。演讲者把这称为“双层限制”：模型层负责“敢不敢说”，客户端层负责“推理之后能不能做”。

这一判断把讨论从“如何让模型回答”推向“如何让工具链持续、可控地运行”。如果一个 AI Coding 系统能读文件、改代码、跑命令、生成 diff、执行测试、发起发布，那么它已经接近一个小型工程操作员。对这类系统，只盯着最终回答太薄，像只看航班到达时间，却不看飞行记录、气象、塔台指令和机组操作。真正需要审计的是从需求进入系统到产物进入仓库、流水线和生产环境的整条链路。

本主题的核心断言

AI Coding 的治理重心正在从提示词审核迁移到行为审核。单个 prompt、单次工具调用、单份日志都只能解释一个碎片；全链路审计要把 prompt、上下文、工具、diff、测试、发布、审批连接成可查询的 Trace。

2 首次定义：审计链路、Trace 与证据 E

2.1 审计链路

审计链路指把一次 AI Coding 行为中的需求、输入、上下文、模型决策、工具调用、文件变更、测试输出、审批记录与发布结果串联起来的证据链。它关心的核心问题有四个：

1. 谁或什么触发了行为：用户、自动任务、外部事件、定时器或上游 agent。
2. 系统依据什么做出行动：prompt、策略、上下文、配置、知识路由、工具描述。
3. 行动造成了什么变化：命令、API 调用、文件 diff、测试结果、构建产物、发布状态。
4. 哪个环节承担责任：模型建议、工具执行、人工审批、流水线门禁、发布者确认。

可以把审计链路想成“软件工程的飞行记录仪”。飞行记录仪不会代替飞行员开飞机，却能在事后回答飞行中每个关键时刻发生了什么。AI Coding 的审计链路也不代替开发者做判断，它为复盘、问责和改进提供足够细的事实地基。

2.2 Trace

Trace 是审计链路中一条从需求到产出的完整轨迹。形式化地说，可以把 Trace 记为一个有向序列或有向无环图：

$$Trace = (V, E_d, meta)$$

其中 V 是节点集合，包括 prompt、context、tool call、diff、test、approval、release 等； E_d 是节点之间的依赖边； $meta$ 是时间、操作者、模型版本、工具版本、仓库版本、风险分级等元数据。这里的 E_d 是依赖边，为避免和证据符号 E 混淆，正文后续用“边”称呼它。

线性任务可以记录为时间序列：需求进入、模型规划、读取文件、编辑文件、跑测试、提交变更。复杂任务更适合 DAG：一个需求会分叉到多个文件、多个工具、多个测试集合，再汇合到一次审批或发布。DAG 的好处是能说明“这个发布依赖哪些证据”，也能说明“哪个失败证据阻断了发布”。

2.3 证据 E 的生命周期

证据 E 是可用于解释 AI Coding 行为的记录集合，包括日志、截图、模型响应、工具调用参数、命令输出、diff、测试报告、审批记录、发布单、回滚记录等。证据生命周期可分为四步：

$$E_{life} = Collect \rightarrow Link \rightarrow Store \rightarrow Query$$

- | | |
|-------------------|---|
| 采集 Collect | 在行为发生时记录事实，避免只在事后补写故事。采集对象至少包括输入、上下文摘要、工具调用、文件变更、测试输出、审批动作。 |
| 关联 Link | 把孤立记录绑定到同一 Trace。比如一次工具调用要关联发起 prompt、执行目录、凭据边界、输出文件和后续 diff。 |
| 存储 Store | 将证据写入可检索、可保留、可校验的介质。高风险动作需要更长保留期和更强完整性校验。 |
| 查询 Query | 支持按需求、文件、工具、风险、操作者、时间、发布版本反查。查询能力决定复盘的速度，也决定问责能否落到事实。 |

3 全链路视图：从需求到发布的 Trace

本主题中，演讲者多次强调“完整日志”“回顾三层到底哪一层出了问题”“把黑箱限制层转化为可审计平面”。这说明审计对象要从单个模型回答扩展到整个 workflow。下面

的流程图给出一条标准 AI Coding Trace。

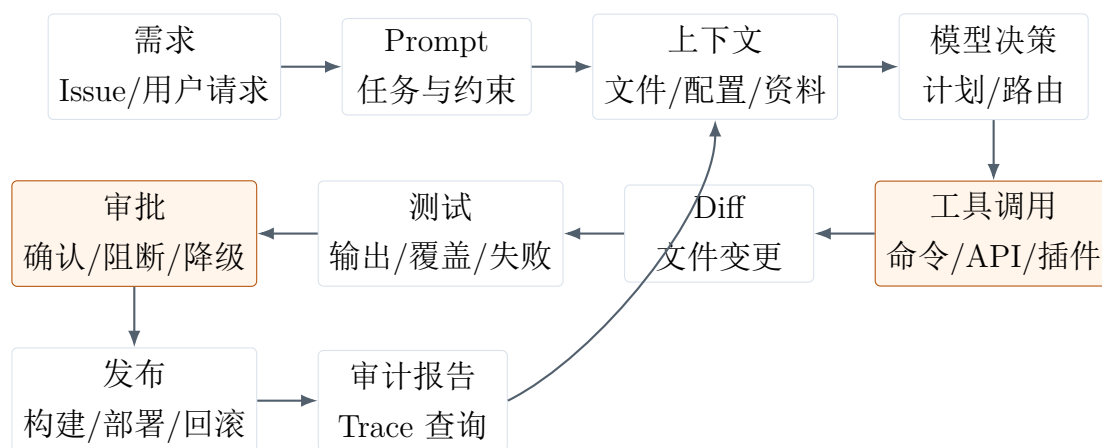


图 1: 09 主题重绘机制图：AI Coding 全链路审计 Trace。图中红色节点代表高风险动作，需要更细的证据采集和审批门禁。

4 审计对象：prompt、上下文、工具、diff、测试、发布、审批

4.1 Prompt：意图与边界的起点

Prompt 是 Trace 的第一类证据。它不只记录用户问了什么，还要记录系统补充了什么约束、策略引擎注入了什么流程指引、知识路由匹配了什么参考材料。演讲中提到“最小注入上下文”：只在需要时注入场景声明、工作流程或行为边界，避免上下文过载。审计层面需要记录这件事，因为同一个用户请求，在不同上下文注入下会导向不同的模型行为。

建议字段包括：原始用户输入、系统提示摘要、策略模板编号、知识路由命中项、模型版本、温度等推理参数、是否触发安全研究场景识别。

4.2 上下文：模型看到的世界

上下文包括代码片段、配置文件、官方参考、历史对话、工具返回、仓库状态。它像模型的“案卷袋”：案卷不完整，结论就可能偏；案卷混入错误材料，结论会被污染。OpenAI 官方资料对 prompt injection 的说明强调第三方内容进入上下文后可能诱导模型执行未授权动作；因此上下文审计要区分可信来源、不可信来源和工具返回来源。

对本主题，官方资料只能来自 `official_references.md`。可引用的官方锚点包括 Anthropic 的 *Beyond permission prompts: making Claude Code more secure and autonomous*,

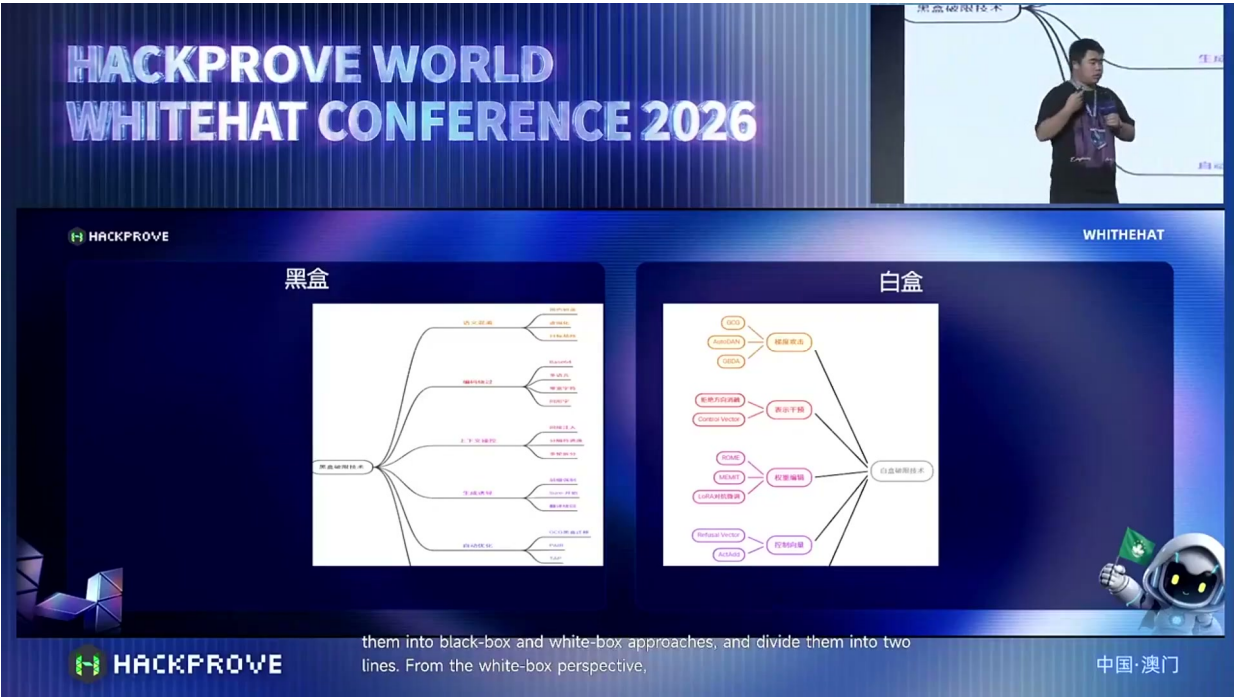


图 2: 黑盒与白盒视角对照: 该截图把不可见模型行为和可观察工程链路放在同一页, 适合引入审计链路的基本问题。审计链路像航班黑匣子, 记录需求、输入、决策、工具、文件变化与审批记录, 后续复盘才有可追溯的证据 E 。¹

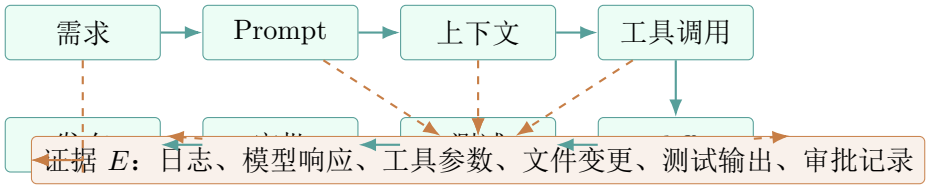


图 3: 审计链路 DAG: 需求进入 Prompt 与上下文, 随后触发工具调用、diff、测试、审批和发布; 证据 E 贯穿每个节点。DAG 的含义是有向无环图, 箭头只沿执行因果前进, 便于把一次 AI Coding 行为还原为可查询的 Trace。²



图 4: OneGod Stack 全景页：截图展示模型、策略、审计日志等层次，适合承接“每一层都要治理”的论点。工程审计的重点是把黑箱行为拆成层级记录：模型层看意图，工具层看权限，日志层看证据闭环。³



图 5: 主题收末页：画面从技术演示回到大会现场，可作为审计流程章节的过渡图。它提示读者，审计链路最终要服务于团队协作、复盘和问责；Trace 的价值在于让复杂工具链留下可复查的因果记录。⁴

以及 OpenAI 的 *Designing AI agents to resist prompt injection*。

4.3 工具：从“说”到“做”的跨越

工具调用是 AI Coding 审计的高风险节点。模型生成建议时，风险仍停留在文本层；工具执行命令、访问文件、调用 API、改写仓库时，风险进入系统层。演讲者的判断很直接：真正危险的点已经从“误导模型”转向“让它执行什么动作”。这和 Anthropic 官方关于 Claude Code 安全的讨论一致：权限提示容易造成 approval fatigue，更可靠的办法是把文件系统、网络和执行边界隔离清楚，在越界时触发审批。

工具审计至少记录：工具名、版本、参数、工作目录、输入来源、输出摘要、退出码、读写文件列表、网络访问状态、凭据使用边界、风险分级、是否人工确认。

4.4 Diff：工程事实的主证据

Diff 是代码层面的主证据。它回答“系统到底改了什么”。审计中要把 diff 绑定到需求、prompt、工具调用和测试输出。没有这种绑定，diff 只能说明文件发生变化，无法说明变化为什么发生、由谁触发、经过哪些验证。

一个可复盘的 diff 记录应包含：变更文件、函数或配置项、增删行摘要、生成方式、人工编辑标记、关联测试、关联审批、风险标签。对安全工程任务，diff 还要标注是否涉及权限、认证、网络、凭据、发布脚本、数据处理逻辑。

4.5 测试：把“可运行”变成证据

测试输出是 Trace 中最容易被低估的证据。它不仅说明功能是否通过，还能说明系统是否遵守了边界。比如一次 AI 生成的修复如果只跑了单元测试，没有跑安全回归测试，审计报告就必须明示验证范围。

建议把测试证据拆成三层：快速反馈测试、风险相关测试、发布前门禁测试。数学上可以用一个覆盖集合表示：

$$Verify(Trace) = T_{unit} \cup T_{integration} \cup T_{security} \cup T_{release}$$

其中每个 T 都应记录命令、环境、输出、失败原因和处理结果。证据缺口要显式保留，不能被“全部通过”四个字糊掉。

4.6 发布与审批：责任落点

发布是 Trace 进入真实环境的边界。审批是责任从自动系统转向人类或组织机制的节点。OpenAI 官方关于 agent 抵抗 prompt injection 的建议强调 source-sink 分析：要把攻击源、敏感汇点和危险动作连起来。对应到发布流程，发布、凭据读取、生产数据访问、外部通知、不可逆迁移都属于敏感汇点，需要确认或阻断。

审批记录应回答：谁批准、批准什么、基于哪些证据、是否有例外、例外是否过期、失败后如何降级或回滚。审批如果只留下“已同意”，价值很低；审批要和 Trace 中的证据节点相连，才能成为问责材料。

5 黑匣子式追溯：把不可见的限制层摊平成平面

演讲者用“黑盒”和“白盒”来解释不同层面的限制。对白盒路径，研究者可能直接分析模型内部机制；对黑盒路径，研究者只能观察输入、输出、拦截、工具行为和客户端策略。AI Coding 产品众多，每套产品的推理机制、策略边界和工具权重都不同，单靠猜测很快会失效。

黑匣子式追溯的思想是：即使无法看到客户端内部实现，也要通过完整 Trace 记录外部可观测行为，把黑箱限制层转化为可审计平面。它像医生看不见人体内每个分子，却可以通过体征、影像、化验和用药反应建立诊断链。AI Coding 审计也是这样：记录输入、上下文、工具请求、阻断原因、放行策略、测试结果和发布状态，就能反推出哪一层影响最大。

追溯问题	需要的证据	可复盘结论
模型误判	原始 prompt、注入上下文、模型响应、拒绝原因	场景识别是否不足，知识路由是否缺失
客户端阻断	工具调用、客户端策略、审批弹窗、退出码	阻断来自工具权限、目录边界或风险分级
工具越界	参数、工作目录、读写文件、网络访问、凭据边界	是否触发敏感汇点，是否需要确认或降级
测试缺口	测试命令、覆盖范围、失败日志、跳过原因	产物是否具备发布证据，缺口是否可接受
发布责任	审批人、发布单、构建编号、回滚记录	谁基于哪些证据放行，事故后如何追责

6 落地模板：字段、grep 词与报告格式

6.1 审计字段模板

下面的字段模板适合落在 JSONL、数据库或日志系统中。关键点是每条记录都要有 `trace_id`，每个节点都要有 `node_id` 和 `parent_ids`。

```
trace_id:      2026-05-18-topic09-demo-001
node_id:       tool-0007
parent_ids:    [plan-0003, context-0005]
node_type:    tool_call
timestamp:     2026-05-18T15:20:31+08:00
actor:        ai-agent
model:        model-name-or-client
tool:         test-runner
workspace:    repo/path
input_digest:  sha256:...
output_digest: sha256:...
risk_level:   medium
decision:     allow | confirm | degrade | block
evidence_refs: [diff-0012, test-0009, approval-0002]
```

6.2 grep 词与风险词

审计报告要保留机器可检索入口。适合本主题的 grep 词包括：

- 行为对象：prompt、context、tool_call、diff、test、release、approval。
- 风险动作：credential、secret、API key、network、deploy、rollback、write。
- AI 安全语境：prompt injection、source-sink、sandbox、tracing，以及审批疲劳。
- 中文审计词：上下文、工具调用、凭据、审批、放行、确认、降级、阻断、复盘、问责。

风险词不等于违规词。它们更像烟雾报警器：报警意味着需要查看证据，不意味着事故已经发生。误报也要记录，因为误报会影响工程体验，最终会诱发绕过治理的冲动。

6.3 报告格式

一份可用的 AI Coding 审计报告建议包含六段：

1. 任务摘要：需求、目标、时间、参与者、模型和客户端。
2. Trace 总览：节点数量、关键路径、分叉点、阻断点。
3. 证据清单：prompt、上下文、工具、diff、测试、审批、发布。
4. 风险判断：敏感汇点、凭据边界、外部写入、生产影响。
5. 验证结果：测试范围、通过项、失败项、未验证项。
6. 问责结论：责任节点、后续修复、策略调整、保留期限。

7 官方参考如何支撑本主题

本讲义没有引入官方参考池之外的材料。两条官方资料对本主题的作用如下：

- Anthropic *Beyond permission prompts: making Claude Code more secure and autonomous*. <https://www.anthropic.com/engineering/claude-code-sandboxing>: 用于说明 AI Coding agent 的安全不能只靠反复弹窗，文件系统隔离、网络隔离、沙箱边界和越界审批是更工程化的控制点。
- OpenAI *Designing AI agents to resist prompt injection*. <https://openai.com/index/designing-agents-to-resist-prompt-injection/>: 用于说明 source-sink 分析、能力最小化、敏感动作确认等思路，适合放进工具调用、发布和审批审计。

8 结语：从单点审核到工作流审计

本主题的价值不在于某个补丁工具本身，而在于它把 AI Coding 的治理对象从“单次回答”推到了“持续运行的工具链”。演讲者最后的判断很硬：过去可能只审核单个工具调用，现在审核要迁移到整个工作流。原因很简单，AI Coding 的风险已经进入行为层，行为层就需要行为级证据。

审计链路、Trace 和证据 *E* 生命周期构成这套方法的三根骨架。审计链路回答“哪些环节要被记录”；Trace 回答“这些环节如何连成一条可追溯轨迹”；证据 *E* 生命周期回答“记录如何采集、关联、存储、查询”。三者合起来，才能让 AI Coding 行为可追溯、可复盘、可问责。